

AIC·AI+开源技术报告

- 团队名称：（报名系统内填写）
- 团队编号：（报名系统内填写）
- 作品名称：Icarus 智测
- 参赛方向：（二）AI 开发工具与开源协作
- 日期：2026 年 月 日

作品速览

要解决的问题	写得出 RTL，写不动测试台；AI 能代写 testbench，但它同时会编期望值——期望值一旦是错的，就会把错的设计判成通过。让被考核的对象兼任考官，是这类做法最根本的缺陷。
我们怎么做	把"提出"与"裁决"分成两个可分别审计的部分：AI 只产出受严格 Schema 约束的测试计划，PASS/FAIL 只由开源仿真器 Icarus 的真实编译与仿真给出；AI 写的期望值被独立的确定性参考模型复算并覆盖。
做到了什么	15 案例 / 83 缺陷 83/83 检出、0 误报、0 不可判定；98 变体 98/98 可综合；离线无密钥跑通完整规划与执行路径；当前回归结果见 docs/experiment/local_validation_2026-10-04.md。
可复核的消融证据	三组实测与一项并集推导（同一批 83 缺陷）：离线规则计划加入参考模型期望值后，检出从 0 → 76；无期望值组没有可比较检查项。该实验检验预言机作用，不证明 LLM 增益（\$5.3）。
开放成果	Apache-2.0；CLI + 库 + 网页 + 可复用 GitHub Action；基准集与 44 条带正反例的规则；《开源及第三方资源使用清单》。
如实说明	不做时序签核/形式化/覆盖率；参考模型只覆盖内置案例；对象③④需求未调研（\$1.0、\$7.4）。

- 说明：本文按《AIC·AI+开源技术报告参考大纲》九节组织，正文超过建议的 15 页，
- 当前页数与体积见本地验证记录。多出的篇幅用于消融实验、三个真实事故复盘、测试条件表与
- 对比选型表，而大纲明确写了"不以报告篇幅、图片数量或材料数量作为评分依据"。
- 更细的过程材料（逐条根因、完整时间线、漏检明细）已移入附录。正文每一项数字都可在随附
- 仓库复现（\$9.9）。评审材料已按竞赛规则第三条第（九）款做匿名处理。

一、作品概述

1.0 作品服务谁（先说清楚，后面所有取舍都从这里来）

一个工具能服务谁，取决于它把什么能力暴露出来了。本项目按"要完成的任务"识别出四类对象，两类主打：

对象	要完成的任务	为什么我们合适
① 用 AI 生成 RTL 的开发者（主打）	"大模型给的 Verilog 能编译，但它到底对不对？"今天的做法是让同一个模型再写一遍 testbench——让被考核的对象兼任考官	第三门就是为"AI 的数字不能作数"设计的：参考模型复算覆盖 AI 期望值、三态证据等级、四层结论口径
② 开源 RTL 维护者 / 贡献者（主打）	"这个 PR 重写了模块，行为变了吗？可上游根本没有 testbench"	verify-diff 不需要被测项目自带 testbench：从 module 头自动提合约、生成计划，两份 RTL 跑同一份计划后逐项 + 逐拍比对
③ 数字电路 / Verilog 学习者	"测试台写不对，报错也看不懂"	完全离线、无需密钥、有中文手册与 15 个可对照案例；失败被翻成"下一步看哪一行"
④ 课程教师 / 助教	"50 份作业要批，判定要一致，学生不服要能解释"	确定性判定 + 逐条证据 + 批量批改与相似度线索 + 全程离线（不把学生代码传云端）

为什么主打 ①②：它们共用同一套底层能力（权威预言机 + 免 testbench 的行为等价对比），所以为一个对象做的适配几乎原样服务另一个；而它们又正好分别对应本赛道的两个关键词——"AI"与"开源"。对象 ③④ 用同一套能力的不同包装覆盖。

诚实标注（未达标项前置）：

对象 ③④ 的用户需求来自真实试用反馈与课程场景推断，尚未做正式调研，本报告不对其需求强度下结论；

对象 ④ 的批量批改已实现（iverilog-ai grade：批量行为对比 + 两两相似度线索表），但没有在真实班级里试用过，因此不声称它在 50 人规模下的实际体验；

正文超过建议的 15 页（当前页数见本地验证记录）；

上游 Issue / Pull Request 尚未提交（见 §6.3）；

其余未达标项集中列在 §7.4。

1.1 背景与问题

围绕 Icarus Verilog 做验证的人，几乎都会卡在同一处：写得出 RTL，写不动测试台。

我们在自己的数字电路学习与项目实践中反复遇到三个具体问题：

（1）测试台的门槛高于设计本身。一个 30 行的模十计数器，要覆盖复位、使能保持、回绕三类行为，手写测试台通常需要 80 行以上，而且每换一个设计就要重写一遍。初学者

的常见结果是"RTL 写完了，但只做了肉眼检查"。

(2) "AI 说验证过了"无法核验。通用大模型可以直接生成 testbench，但它同时会生成期望值——而期望值一旦是错的，错的期望值会把错误的设计判成"通过"，或者把正确的设计判成"失败"。这不是假设：本项目在真实在线模型实验中就观测到过这类现象（详见 §5.3）。让被考核的对象同时担任考官，是这套做法最根本的缺陷。

这一条在 2026 年变得比以往更迫切：AI 已经能写出一份能编译、看起来合理的 RTL，唯一的缺口是"没人验"。而验收一份 AI 改写的 RTL，恰恰是最需要"比两份实现"而不是"从头写一个测试台"的场景——这正是对象 ①② 共用的那套能力。

(3) 现有工具链缺一自动化。Icarus Verilog 是成熟、可靠、完全开源且可脚本化的仿真器，它的定位是"给定 testbench，给出结果"。从"我想验证什么"到"可执行的 testbench"这一段，目前没有开源工具覆盖。上游开源 RTL 项目普遍没有测试台，也就没有任何低成本的行为回归手段。

1.2 目标、参赛方向与作品形态

参赛方向：(二) AI 开发工具与开源协作。

作品形态：命令行工具 + 网页应用 + GitHub Action + 可复用基准集与规则包。

核心功能：把自然语言的验证目标，转换为受严格约束的结构化测试计划，再由确定性代码生成 testbench，交给 Icarus Verilog 真实编译与仿真，最后产出可追溯的证据与报告。针对对象 ②，另有一条不需要输入测试台的路径：从 RTL 自动提合约、生成计划，把同一份计划喂给两份 RTL，判定行为是否一致。

一句话概括本项目的分工主张：AI 提出测试假设，开源仿真器作出判决。

1.3 核心亮点

AI 不参与判分。PASS/FAIL 只由 Icarus 的编译/仿真结果与结构化断言决定；AI 的期望值被独立的确定性参考模型复算覆盖，不一致只记诊断，不影响结论。

25 案例 / 83 可复现缺陷，实跑 83/83 检出、0 误报、0 不可判定 (§5.3)。

证据分层，并把"没做的"写出来。五层证据表把"时序分析 / 布局布线 / 上板验证"显式标"未运行"，避免把"仿真通过"或"综合通过"误读为"能上板" (§5.5)。

离线可完整复现。自带离线调试模型服务（只监听回环、不需要密钥、不联网），评委可在无凭据环境下跑通全部流程与实验。

验收一份 RTL 不需要先写测试台。iverilog-ai verify-diff 一条命令判断两份 RTL 行为是否一致（合约与计划都可自动准备），产出可直接贴进 PR 的对比报告。

结论口径分四层、措辞互不重叠。运行状态 / 比对情况 / 设计结果 / 对比结论各用一套不共用的词，避免同一个 passed 被读成三件事 (§4.2)。

二、需求分析与总体方案

2.1 需求分析

(1) 现有做法与痛点

现有做法	主要痛点	需求优先级
手写 testbench	工作量大、边界易漏、换设计即重写	覆盖度
通用大模型直接写 testbench	期望值不可信；无法离线复核；结论不可追溯	可信度
商业仿真器的 AI 助手	闭源、需授权、无法审计判决路径	可信度
上游项目"没有测试台"就放弃回归	重构与 AI 改写无法验收	可信度

我们的优先级排序是 可信度 > 覆盖度 > 效率：一个"看起来自动化了但结论不可信"的工具，在验证场景里比没有工具更危险。

(2) 需求量化（每一行都对应一处可核验的实现）

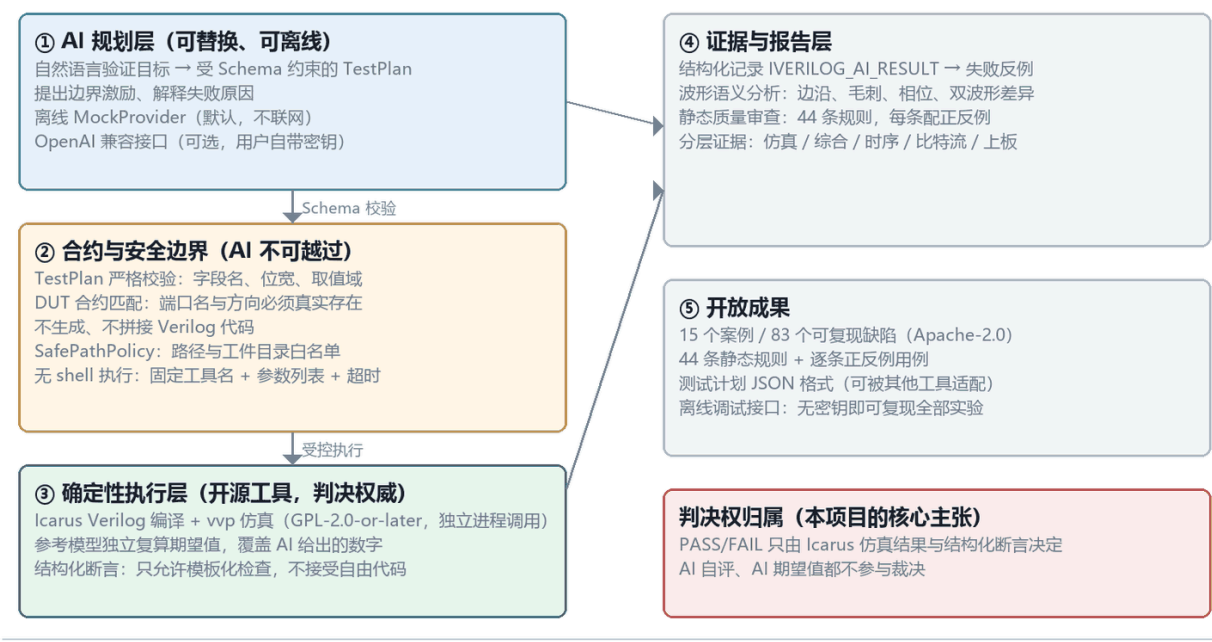
需求	目标	说明
结论必须可信	AI 期望值一律由独立参考模型复算覆盖	第三门；SUPPORTED == AUTHORITATIVE 有断言守着
结论必须可读	四层措辞互不重叠，不出现"同名不同义"	core/labels.py 为唯一措辞来源
不要求用户先写测试台	从 RTL 自动提合约、按合约生成测试计划	verify-diff 默认路径
判定必须可复现	重放同一输入与测试计划 → 同一结论；退出码稳定	当前回归结果见 docs/experiment/local_validation_2026-10-04.md；5 个矩阵脚本
判定必须可追问	每条失败落到"第几拍 / 期望 / 实际 / 失败指纹"	结构化记录 + 失败解释
成本学生可承担	全流程本地、离线可跑；在线开销透明	无密钥跑通完整 AI 路径；费用按核验价目表给区间
能接进别人的流程	命令行退出码 + GitHub Action	verify-diff 的 0/1/2；action.yml

(3) 未达标项（如实列出）

- 对象 ③④ 的需求未做正式调研，本文只用试用反馈与课程场景推断；
- 对象 ④ 的批量批改已实现（iverilog-ai grade），但没有在真实班级里试用过；
- 正文超过建议的 15 页（当前页数见本地验证记录）；
- 上游 Issue / Pull Request 尚未提交（见 §6.3）。

2.2 总体方案

图 1 系统架构：AI 能影响什么，不能影响什么



AI 只能产出行 ① 的结构化测试计划；②③ 两层是确定性代码，AI 无法改写执行内容与判决口径。

图 1 系统架构

系统分四层，关键设计是把 AI 关在第一层里。各层职责如下：

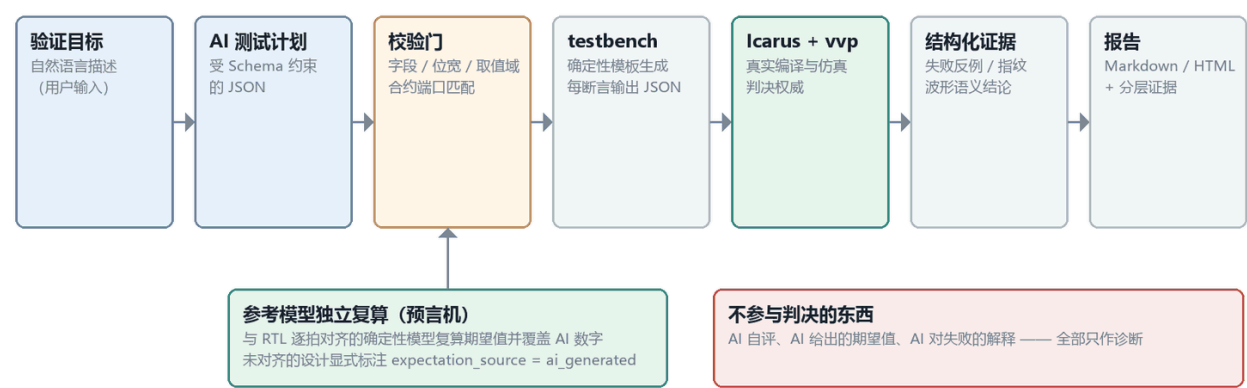
层	职责	AI 能影响吗
① AI 规划层	只输出结构化 JSON 测试计划；离线时用按合约生成激励的确定性规则引擎，需要真实模型能力时接任意 OpenAI 兼容接口	能（唯一——层）
② 合约与安全边界	字段/位宽/取值域严格校验 + 与 DUT 合约端口逐一对齐；不生成、不拼接 Verilog；路径白名单；执行不经 shell	不能
③ 确定性执行层	Icarus 编译 + vvp 仿真作为唯一判决权威；参考模型独立复算期望值并覆盖 AI 数字	不能
④ 证据与报告层	结构化记录、波形语义分析、静态质量审查、分层证据表、四层结论口径	不能

一次验证的数据流（10 步，每一步的产物都可单独审计）：

步骤	发生什么
1	用户给出验证目标（一句话）与 DUT 合约（端口/时钟/复位）
2	AI 规划层据此产出 TestPlan JSON：每条向量写明驱动哪些端口、跑几拍、期望什么

3	门 1：严格 Schema 校验（字段名、类型、位宽、取值域）——杜撰字段或越界取值直接拒绝
4	门 2：与 DUT 合约逐端口对齐——端口名或方向对不上直接拒绝
5	门 3：参考模型按同一份激励独立复算期望值，覆盖 AI 给出的数字
6	门 4：结构化断言只允许五种受限模板，模型无法注入 Verilog/SVA 代码
7	确定性模板生成 Verilog-2001 testbench；每条断言输出一行 IVERILOG_AI_RESULT {json}
8	Icarus 编译 → vvp 仿真（固定参数列表、不经 shell、带超时与输出上限）
9	严格解析器消费结构化记录；结构化断言在采样序列上判定；波形语义分析读 VCD
10	产出报告（四层结论口径 + 证据等级）、失败反例（带失败指纹）、VCD 与证据包

图 2 验证流程：从自然语言验证目标到可核验报告



所有结论可回溯到 run 目录下的编译日志、仿真日志、VCD 与 result.json。

图 2 验证流程

2.3 可行性与成本

学生团队可承担——这是本项目的一条硬约束，每一项都有对应的实现：

维度	数据	结论
算力	不需要 GPU、不需要训练模型；全部实验在本机 CPU 完成	普通笔记本即可

软件	全部开源免费 (Icarus / Python / pydantic / Streamlit; Yosys 有 WASM 版, 离线可跑)	零授权成本
单例耗时	"生成计划 → 生成 testbench → 编译 → 仿真 → 出报告"秒级; 15 案例 + 83 缺陷全矩阵分钟级	迭代快
离线能力	无密钥、不联网即可跑通完整 AI 路径 (含调试接口)	无网络也能演示
在线开销	历史双模型各 10 轮实验按当时标价估算 \ \$2.6–5.2; 同期余额下降记录 ¥23.66, 未逐请求对账。当前单模型 10 轮的用量与估算见 \$5.3(3)	区分 API 返回用量与费用估算
数据	不需要外部数据集; 基准集由团队自行编写并逐项实跑验证	无数据授权风险

三、AI 技术与开源及第三方资源选择

3.1 AI 技术的作用

AI 在本项目中承担三件事，每一件都配了对应的验证手段——这是本项目与"让模型直接写 testbench"最根本的分工差别：

(1) 把自然语言验证目标转成结构化测试计划。

技术实现：模型的唯一输出是一份 JSON (vectors[].inputs 驱动哪些端口、跑几拍, expected 期望什么) , 受严格 Schema 约束；提示词里内置全部 44 条规则的上下文与实测编码约定。

为什么需要 AI / 怎么验证：边界条件的枚举空间大且依赖设计语义，人工穷举成本高——这正是生成式模型擅长的"按语义提出候选假设"。验证方式是四重门控（下表），以及离线消融对执行链的检验：离线规则计划 + 权威预言机可检出 76/83，无期望值组为 0/83。后者没有可比较检查项；该对照不使用真实 LLM，不能证明模型增益。真实模型结果见 \$5.3(3)。

(2) 提出边界激励。

技术实现：模型在计划里显式写出每条向量的 rationale（人读的意图说明），例如"FIFO 写满" "计数器回绕" "101 序列重叠检测"。

怎么验证：与仓库自带的人工边界激励做同口径对照。实测：通用激励比人工边界激励少检出 7 条，且少检出的全是"要精确时序或边界序列才能触发"的缺陷（\$5.3）——这里比较的是离线规则激励与人工激励，不能直接推断真实 LLM 的能力。

(3) 解释失败原因。

技术实现：把结构化失败记录（检查项 / 第几拍 / 期望 / 实际）翻成"下一步看哪里"，指出该信号在 RTL 里被赋值的行号，并给出复现命令。

怎么验证：措辞上严格守住一条线——只写"是「被赋值的位置」，不一定是「出错的位置」"（问题也可能在激励、复位或上游信号），并有测试钉住这句。

四重门控——AI 输出的验证链（这是本项目的核心工程内容）：

门	机制	拦截什么
第一门	严格 JSON Schema 校验（字段名、类型、位宽、取值域）	格式错误、杜撰字段、越界取值
第二门	DUT 合约匹配（端口名与方向必须真实存在）	模型杜撰端口别名（如把 enable 写成 en）
第三门	确定性参考模型复算并覆盖 AI 期望值	AI 写错期望值导致假通过或假失败
第四门	结构化断言只允许五种模板化检查	模型注入自由 Verilog/SVA 代码

第三门是决定性的。内置案例的参考模型都已与参考 RTL 逐拍对齐（15/15，多个随机种子零差异），因此在这些内置案例中用模型复算值作为权威期望值。离线消融中，有参考模型期望值时检出 76/83；移除期望值后没有可比较检查项，检出为 0/83。

3.2 开源及第三方资源选择

完整的名称、版本、来源、许可证、使用方式、关键义务与自研边界的逐项清单，见随报告提交的《开源及第三方资源使用清单》（附件）。选型依据摘要：

资源	选它的理由	许可证	使用方式
Icarus Verilog	唯一成熟、完全开源、可脚本化、支持 Verilog-2001/-2012 的仿真器	GPL-2.0-or-later	独立进程调用，不分发；判决权威
Yosys / YoWASP	开源综合工具；WASM 版本可完全离线运行	ISC	独立进程调用，不分发；可选证据层
pydantic	严格 Schema 校验，是把 AI 输出关进笼子的关键依赖	MIT	库依赖
Streamlit	零前端成本的单页演示，评委可在浏览器里直接复现	Apache-2.0	库依赖（可选）
GTKWave	可选的人工波形复核	GPL-2.0-or-later	独立进程调用，核心流程不依赖

3.3 自主开发与权利边界

类别	内容
----	----

第三方开源提供	仿真引擎（Icarus）、综合引擎（Yosys）、Schema 校验库（pydantic）、页面容器（Streamlit）。 均为独立进程调用或库依赖，未修改、未分发其代码
通用 AI 工具完成	辅助编写部分 Python 模块、Verilog 测试台与文档草稿；生成内容不主张为团队自主知识产权，且其正确性一律由团队的编译、仿真与测试逐项验证
团队自主设计/开发/验证	测试计划 JSON 格式与校验规则；确定性 testbench 生成器；参考模型与逐拍对齐方法；结构化断言模板；基准集（15 案例 / 83 缺陷）与其边界 testbench；44 条静态规则及逐条正反例；波形语义分析（边沿/毛刺/相位/差异）；分层证据层；行为对比工作流 verify-diff + 可复用 GitHub Action；四层结论口径与措辞表；批量批改与相似度线索；安全边界（路径策略、无 shell 执行、超时与输出上限）；全部中文文档与实验记录

3.4 融合方式：为什么不是"简单调用接口"

AI 输出必须穿过确定性边界才能执行。模型输出必须通过 Schema 与合约双重校验，再由确定性代码生成 testbench；模型从不接触 Verilog 语法层，因此结构上不可能注入代码、命令或路径。

开源仿真器是判决权威，AI 只是假设提出者。即使模型完全不可靠，工具仍然可用（退化为"人工设计激励 + 真实仿真裁决"）。

参考模型独立复算，堵住"写错期望值"这条路。这是同类工具普遍缺失的一环，也是我们在真实模型实验中观测到问题之后专门补上的一环——其作用见 §5.3 的消融实验。

四、设计与实现过程

4.1 核心功能与流程

功能	输入 → 输出
测试计划生成与校验	自然语言目标 + DUT 合约 → 通过 Schema 校验的 TestPlan
确定性 testbench 生成	TestPlan + 合约 → Verilog-2001 testbench（每条断言输出一行结构化 JSON）
执行与取证	testbench + RTL → 编译/仿真日志、结构化记录、失败反例、VCD
权威预言机	TestPlan + 合约 → 参考模型复算的期望值（覆盖 AI 数字）

结构化断言	受限模板（信号相等/稳定/永不置高） → 通过/失败记录
波形语义分析	VCD → 边沿统计、毛刺判定、相位偏差、双波形差异、失败时间窗
静态质量审查	RTL 源码 → 44 条规则命中 + 质量评分 + Markdown 报告
分层证据	RTL 源码 + Yosys → 综合状态、单元统计、五层证据表
行为级对比	两份 RTL + 同一 TestPlan → 逐检查项差异 + 波形差异
行为对比 workflow	两个 RTL 文件 → 自动提合约草稿 + 生成计划 → 结论句 + 差异清单 + 可贴进 PR 的报告（不需要被测项目自带 testbench）
批量批改	一整个目录的提交 + 标准实现 → 逐份结论 + 两两相似度线索（区分"改名抄"与"直接抄"）
可复用 GitHub Action	PR 上 uses: 一行 → 行为回归结论写进 Job Summary / 产物 / PR 评论

4.2 关键实现

每个关键实现都按「技术实现 + 为什么这么做」两段写——后者才是评审真正想看的判断。

（1）确定性 testbench 生成器。

技术实现：生成器不解析、不拼接模型提供的任何文本，只把已通过校验的标量数据填入固定模板；每条断言输出一行 IVERILOG_AI_RESULT {json}，由执行器的严格解析器消费。

为什么这么做：这样"AI 生成的内容"与"可执行代码"在结构上被彻底分离。不是靠提示词叮嘱模型"不要写危险代码"，而是让模型根本没有写代码的通道。

（2）权威预言机门控。

技术实现：只有与 RTL 逐拍对齐的参考模型才能覆盖期望值；未对齐的设计返回空字典，调用方回退为 AI 期望值并如实标注 expectation_source="ai_generated"。

为什么这么做：模型一旦与 RTL 差一拍，就会把"参考设计通过"改写成"参考设计失败"，比没有预言机更糟。放宽这个集合的方法只有一个——让模型逐拍对齐（"对齐一个、加入一个"）。离线消融中，有参考模型期望值时检出 76/83；无期望值时没有可比较检查项，检出为 0/83。

（3）IEEE 1364 非阻塞赋值语义对齐。

技术实现：把同一组向量同时喂给参考模型与 RTL，逐拍比较可观测输出，要求零差异。

为什么这么做：对齐过程中共发现并修掉 8 个模型错误，其中三个同一根因——同一时刻对同一变量的非阻塞赋值，其右侧读到的是旧值。例如 UART 发送器写 bit_idx <= bit_idx + 1 之后同拍读 frame[bit_idx+1]，用的是旧的 bit_idx；模型若用新值取位，输出就整体错开一拍。SPI 的 sclk/mosi、去抖的 key_state 都栽在同一处。这类问题只有逐拍对照才能发现，肉眼看波形很容易误判成"差不多对"。

(4) 路徑與進程安全。

技術實現：所有外部進程以固定工具名 + 參數列表啟動，不經過 shell；輸入與工件路徑走白名單策略；每個進程帶超時與輸出上限；不自動覆蓋原始 RTL。

為什麼這麼做：工具要跑用戶上傳的 RTL，“不執行來路不明的代碼”不能只寫在文檔里——必須讓執行路徑上根本沒有拼接字符串的機會。

(5) 噪聲校準作為交付物的一部分。

技術實現：靜態規則與波形毛刺判據各經歷三輪校準，逐條核實命中是否為真。

為什麼這麼做：初版規則對倉庫內 84 個 RTL 文件報出 317 條命中，含大量誤報

（如把注釋結尾 / 當成除法、把合法寫法誤判為鎖存器）。“規則命中數”本身不是成果，“命中里有多少是真的”才是——所以校準過程也寫進交付物（§5.4）。

(6) 三個真實事故的复盘。三條都來自真實使用（含本人自測與外部試用反饋），每條都能在倉庫提交歷史里找到對應修復與回歸測試。寫進報告是因為“踩了什麼坑、怎麼定位、怎麼防止復發”比一串功能清單更能說明工程能力，也說明判據確實在被真實場景壓測，而不是只在測試集上好看。完整复盘表（症狀 / 根因 / 修復 / 回歸）見附錄 §9.6。

三條各自暴露一類問題：① 離線路徑喂了寫死的演示計劃（選擇分支時沒跟著合約走）；② Streamlit 的 widget 與 session_state 生命週期衝突（先建控件再寫同名鍵必然報錯）；③ 提示詞與校驗器各寫了一份字段表——這一條最值得記：我們一度把它歸因為“模型寫錯字段”，復核後發現提示詞自己就把該字段列進了模板，模型只是照着寫，缺陷在本項目一側。報告按修復後的事實陳述，不把自己的缺陷記成模型能力問題。

4.3 團隊分工與過程記錄

成員分工：成員 A 負責 AI 規劃層與 Provider 适配、測試計劃 Schema 與校驗；成員 B 負責確定性執行層、安全邊界與證據鏈；成員 C 負責基準集與缺陷設計、參考模型對齊與文檔。三人共同完成網頁演示與實驗設計。評審材料按競賽規則第（九）款做匿名處理。

過程記錄：項目以版本迭代方式推進，倉庫提交歷史即完整過程記錄（八個階段，每階段有對應的提交信息與驗證結論），階段時間線見附錄 §9.7。

生成式 AI 與智能體工具的使用：用於輔助編寫部分 Python 模塊、Verilog 測試台與文檔草稿。團隊的審核、修改與驗證工作包括：逐文件人工審核；以本机 Icarus 真實編譯與仿真結果作為功能結論的唯一依據；執行自動化回歸（結果見 docs/experiment/local_validation_2026-10-04.md）；對每個缺陷變體單獨復現失敗證據；核對每條技術陳述與實驗產物的一致性，刪除與實現不符的表述。對所用模型/平台的許可與服務條款：項目默認不請求任何網絡服務，僅在用戶顯式配置密鑰並打開網絡開關後才調用，且不在倉庫內保存憑據。

AI 生成內容的權利核査：倉庫內無第三方複製代碼；未使用第三方圖片、音視頻或數據集；報告插圖由團隊用自有腳本（scripts/make_report_figures.py）程序化生成。

五、成果展示与效果验证

5.1 成果展示

作品提供命令行与网页两个入口。网页包含：AI 接口设置、测试计划查看、执行流水线、失败反例、波形语义结论、分层证据、静态质量审查、自定义 RTL 导入与行为级对比。页面顶部固定展示"实证状态"面板（案例数、缺陷数、规则数、测试数、已对齐模型数），全部数字由页面实时读取仓库文件现算。

复现方式见 §9 附录；网页入口：streamlit run ui/app.py。

5.2 验证方法

方法	做法	能回答的问题
固定矩阵基准	15 个参考设计 + 83 个缺陷变体，逐个跑同一固定激励	能不能检出已知缺陷，会不会误报
消融实验	同一批 83 个缺陷，三组实测与一项并集推导；B/C 保持计划相同，仅改变期望值来源	离线规则计划需要哪些判定条件；不测 LLM 增益
逐拍对齐	同一组向量同时喂给参考模型与 RTL，多随机种子逐拍比较	权威期望值本身是否正确
真实模型预算对齐实验	四种策略跑同一批案例，每计划刺激预算一致；明确各组轮数，并另算 seed 0-4 子集	在限定预算与开发集上的检出差异、波动和成本
负例验证	构造"必须失败"的输入（不可综合 RTL、语义等价重写、已知缺陷、编译不过的候选）	判据是否真的在起作用，而不是永远说"通过"
自动化回归	含实跑 Icarus 的端到端用例 + mypy / 死代码 / 编码损坏门禁；本轮结果见 docs/experiment/local_validation_2026-10-04.md	改动是否破坏既有结论
外部复核	10 分钟复核指南（docs/reproduce_in_10_minutes.md）：5 条命令 + 期望数字 + 对不上怎么反馈	别人能不能自己验一遍

测试条件（每个数字都在这个条件下取得，换条件就要重测；完整表见附录 §9.10）：Icarus Verilog 12.0 (-g2012)、Windows 11 x64 + Python 3.11 (CI 另跑 Linux)、单例超时 30 秒。离线固定种子路径可确定性重放；随机策略须保存 seed，在线模型重新生成计划可能产生不同结果。离线流程不需要密钥或联网；在线实验需要模型服务。超时与编译失败一律记为"不可判定"，不折算成检出。

5.3 验证结果

图 4 基准矩阵实跑结果（固定向量，可复现）



图 4 基准矩阵结果

(1) 固定矩阵基准（可一键复现）

指标	结果
参考设计通过	15 / 15
缺陷检出	83 / 83（检出率 100%）
参考误报	0
不可判定	0

(2) 离线消融：激励与参考模型的作用（python scripts/run_ablation_matrix.py）

上面那张表说明手写 testbench 在内置基准上的检出结果。本节用同一批 83 个缺陷做三组实测，另计算 A 与 C 的并集；不是四档实测，也没有调用真实 LLM。B/C 的计划相同，仅改变期望值来源；A/C 比较手写测试台与离线规则计划两种路径，不能视为只改变一个变量的严格消融。

配置	激励来源	期望值来源	缺陷检出	参考误报	不可判定
A handwritten-tb	人工设计的边界激励（仓库自带 testbench）	人工写在 testbench 里	83 / 83	0 / 15	0
B ai-plan-plain	离线规划器按合约生成	没有期望值	0 / 83	0 / 15	0
C ai-plan-oracle	离线规划器按合约生成	参考模型独立复算并覆盖	76 / 83	0 / 15	0
A U C（推导值，非实测组）	合并 A、C 检出集合	—	83 / 83	—	—

结果与边界：

B. → C：0 → 76。两行的激励完全相同（同一份计划、同一个随机种子），

只把期望值从"没有"换成"参考模型独立复算"。B 组没有可比较检查项，不能检出缺陷；C 组得到 76/83。这说明当前离线执行链需要独立期望值，不等于测得 LLM 的能力提升。

A → C: 83 → 76。离线规则路径少检出 7 条。两条路径都使用确定的期望值，但测试台、激励与观测方式不同。少检出的 7 条都需要精确时序或边界序列：异步复位在两沿之间断言 (mod10_counter)、同步器级数与释放窗口 (sync_reset × 2)、抖动序列长度 (debounce)、展宽期内重触发 (pulse_stretcher)、写满 off-by-one (sync_fifo)、紧急抢占时刻 (traffic_light_emergency)。逐条明细见附录 §9.8。这组对照只支持"当前离线规则路径未覆盖全部人工测试边界"，不外推为真实模型的结论。

A ∪ C = 83/83，规划器独立贡献 0 条。这个结论对我们不利，但必须如实写：当前离线确定性规划器的价值不在"发现人没想到的缺陷"，而在"不用人手写 testbench 就能 拿到 76/83"。真实模型的实验另列于 §5.3(3)，不能用本节离线消融替代其能力证据。

参考模型对齐：15/15 个内置案例的参考模型与 RTL 在 4 个确定性随机种子上逐拍零差异，因此全部可以作为权威预言机；建模覆盖与权威集合完全一致（有测试断言防止放宽）。

(3) 预算对齐后的重跑与同轮数子集（每计划刺激预算一致、固定分母、参考告警作废）：

旧的比较不成立——三组各自决定跑多久（实测同案例 fixed/random/ai/online 分别是 4/12/12/18 个向量），分母还由"实际产出的行"决定，请求失败会让分母一起缩小。改造后：每个案例的总刺激周期先按规格定好（见表下方说明），各策略共用同一个值；计划执行前归一化（超长尾部截断、不足按末态保持补齐）；分母来自 benchmarks/manifest.json 登记的（案例, 变体）；计划生成失败/编译失败/超时/零可比较检查项一律记"不可判定"并按未检出计入；参考设计自己出现功能不一致时，该（案例, 轮次）的变体检出作废。

原始记录的轮数不相等：fixed 为 1 轮，random/Mock 各 5 轮，在线模型为 10 轮（seed 0–9）。因此“每计划同预算”不能写成“多轮并集的总预算相同”。以下完整保留原始结果：

策略 / 轮数	分母	累计并集检出	单轮平均	单轮范围	不可判定变体 runs	参考告警作废变体 runs
固定向量（人工），1 轮	67	57 (85.1%)	85.1%	85.1%–85.1%	0	0
随机向量，5 轮	67	57 (85.1%)	78.2%	74.6%–83.6%	0	0
MockProvider，5 轮	67	57 (85.1%)	78.2%	74.6%–83.6%	0	0
在线模型（deepseek-flash），10 轮	67	63 (94.0%)	83.4%	73.1%–89.6%	8	5

原始文件 .iverilog-ai/strategy-online-flash/strategy_matrix.json 共 1,680 条 runs，其中在线组 800 条（13 个参考设计 + 67 个缺陷，每轮 80 条），对应 13 案例 × 10 轮 = 130 次计划请求；其余为 fixed 80 条、random 400 条、Mock 400 条。代码版本 47ed59b9，工作区干净，2026-10-01T10:41:57Z→11:57:07Z。在线不可判定 8 条中，5 条因参考告警作废，3 条因计划校验失败未产生比较记录（汇总记为 no_records）；这些均是 run 级计数，不是 8 个独立计划。十轮在线并集 94.0% 不能直接证明其在同总预算下优于五轮随机或单轮固定激励。

为比较相同轮数，2026-10-04 从上述原始文件事后选取连续 seed

0-4，未重新调用模型，未改写原始记录：

策略 / 子集	累计并集检出	单轮平均	单轮范围	不可判定 / 参考告警作废 runs
随机 / Mock, 各 5 轮	57/67 (85.1%)	78.2%	74.6%–83.6%	0 / 0
在线模型, 5 轮	61/67 (91.0%)	83.9%	79.1%–86.6%	0 / 0
固定向量, 原始 1 轮参照	57/67 (85.1%)	85.1%	85.1%–85.1%	0 / 0

重算证据：docs/experiment/matched-budget-2026-10-04/summary.json 与 selected_runs.json；脚本 scripts/summarize_matched_budget.py。在线与随机/Mock 在该五轮子集下每计划预算与轮数一致，在线累计净多检出 4 条，单轮平均高 5.7 个百分点；83.9% 仍低于人工固定向量的 85.1%。原始十轮的单轮均值 83.4% 也低于 85.1%。固定组实际只跑一轮，不据此宣称在线的多轮并集在同总预算下优于人工。这是同一开发集上的事后描述性分析，未做独立留出评测，不支持泛化或显著性结论。MockProvider 回放随机计划，不是独立的 LLM 能力证据。

用量记录与费用估算：原始在线十轮共 130 次计划请求，其中 128 个请求保存了 API 返回的 usage；按 request_id 去重后为 prompt 242,146、completion 1,000,332 tokens（含 reasoning 787,217）。另外 2 个请求未保存 usage，不能据此断言它们未计费。按当时记录的 deepseek 标价估算，高峰 cache-miss 情景约 \$1.273、非高峰约 \$0.637；这是基于已记录 token 的估算，不是实付账单。五轮子集 65 次请求中 63 个有 usage，prompt 119,262、completion 470,098（含 reasoning 367,668），合计 589,360 tokens；不把十轮费用挪作五轮成本。逐项来源与精确值见 docs/experiment/metric_inventory.md。

预算取值（src/iverilog_ai/core/strategy_scoring.py 的 CASE_BUDGETS，按规格而非按成绩定）：UART 40 周期（要覆盖起始位+8 数据位+停止位）、SPI 64、FIFO 40、消抖 32、PWM 32、状态机类 24、组合类 16–20。同案例各策略的每个计划刺激周期一致（按 total_cycles 核对），不代表不同轮数组的累计刺激预算一致。（4）真实在线模型实验（旧口径，探索性）——13 案例 / 67 缺陷 × 每案例 10 次重复 × 2 个模型，每个模型 130 次真实请求；详见 docs/experiment/model_comparison_2026-09-12.md。下表给出单轮平均检出率（括号内为 10 轮累计并集）：

必须先读这一条：这一轮跑在旧口径下——各组的总刺激周期并不相等，分母由"实际产出的行"推导，compile_failed 不计入不可判定，参考设计自己报警也不作废该轮检出。因此它是探索性结果，不能与本报告 §5.3(3) 的预算对齐实验数字并列比较或相加。真实模型在新口径下的原始十轮与五轮事后子集结果见 §5.3(3)，两处数字口径不同、不可互相印证，只作为趋势参考。原始文件只读保留在 .iverilog-ai/model-compare-r10-/；逐项口径与来源见 docs/experiment/metric_inventory.md。公平口径的改造与重跑（同预算、固定分母、参考告警作废）见 §5.3(3)。

策略	单轮检出率	累计并集	说明
固定向量（1 轮）	83.6%	56/67	人工设计的固定激励
随机向量（1 轮）	76.1%	51/67	同种子可复现的随机激励

离线 AI 规划（1 轮）	76.1%	51/67	确定性离线模型，用于验证流程而非能力
在线 AI（flash，10 轮）	85.8%	65/67（97.0%）	比固定向量高 2.2、比离线基线高 9.7 个百分点
在线 AI（v4-pro，10 轮）	87.8%	64/67（95.5%）	单轮更稳，但累计并集略低；生成慢 3.4 倍

该轮参考误报为 0（125 次参考设计运行无一次硬失败）。同时公开的未达预期项：2 个缺陷未被检出，原因已定位且都不是模型能力问题，而是激励相位/观测点——这两条在 手写边界 testbench 的固定矩阵里都能检出；另有 5 次请求（3.8%）的计划被严格校验拒绝，按"不可判定"如实记录，不折算成检出。其中一次拒绝的根因经复核在本项目一侧（提示词与校验器各写了一份字段表），已修复并加回归门禁，详见附录 §9.6 事故 3。

费用估算：\\$.2.6–5.2（两模型合计，按当时记录的官方标价换算，scripts/estimate_model_cost.py），同期 API 余额下降记录为 ¥23.66，未逐请求对账，不能将估算当作实付费用；两模型的漏检集合几乎不重叠（并集 66/67），多模型的收益来自互补而非选最强。

（5）负例验证（证明判据不是"永远通过"）：

不可综合的 RTL（变量上界的 while 循环）→ 综合层报 失败；

语义等价但写法不同的重写（多出一个内部辅助信号）→ 行为对比判 一致；

已知缺陷（极性反转、计数差一）→ 行为对比判 不同，并定位到具体信号与拍；

候选编译不过 → 行为对比判 未取得可比证据（退出码 2），而不是"行为不同"——这两件事必须分开：前者是"没跑起来"，后者是"跑起来了但结果不同"。

（6）尚达到不了的目标：不做时序签核、不做代码覆盖率、不做 CDC 形式化、参考模型只覆盖内置案例——完整边界清单见 §7.4。

（7）量化成果指标（收尾总表：每一行都能在仓库里复现）

维度	数据	成果
正确性	15 案例 / 83 缺陷，83/83 检出、0 误报、0 不可判定	具备可核验的缺陷检出能力，且不靠猜
可信度	离线规则计划加参考模型期望值检出 76/83；无期望值组为 0/83	验证独立期望值在当前执行链中的作用；不证明 LLM 增益
可复现	当前回归结果见 docs/experiment/local_validation_2026-10-04.md；5 个矩阵脚本	离线固定种子路径可重放；在线计划须保存后复跑
可解释	每条失败落到"第几拍 / 期望 / 实际 / 失败指纹"；失败被翻成"看哪一行"	结论可追问，学生/评审都能自己看
成本	离线路径无模型调用费；历史双模型各 10 轮按当时标价估算 \\$.2.6–5.2	用量、估算费用与实际账单分开披露

开放	Apache-2.0; CLI + 库 + 网页 + 可复用 GitHub Action; 基准集与规则包	他人可直接复用, 含 PR 级行为回归
边界	不做时序/形式化/覆盖率; 参考模型只覆盖内置案例; 范围之外明确不说	不夸大——把没做的写在同一张表里

5.4 噪声校准记录（为什么这部分也算成果）

静态规则与波形毛刺判据都经历了三轮校准才做到不误报：规则从"9 条规则报出 317 条命中、含 error 级误报"收敛到"44 条规则、163 条命中、0 条 error 级误报"；毛刺判据从"窗口内翻转多次即不稳定"（在时钟上全面误报）改为"全局基线 + 连续 ≥3 个异常间隔 + 复位沿不参与统计"。逐条根因见附录 §9.11。

我们把这两段过程写进报告，因为"规则命中数"本身不是成果，"命中里有多少是真的"才是。

5.5 分层证据

图 3 分层证据：做过的与没做的都写出来

功能仿真	本次流水线提供	Icarus 编译 + vvp 执行 + 结构化断言
逻辑综合	已提供（可选）	Yosys 通用门级映射 + 单元统计（实测 98/98 个 RTL 变体通过）
时序分析	未运行	需要目标器件时序库与时钟约束 —— 本项目不提供
布局布线 / 比特流	未运行	需要厂商工具链（Vivado / Quartus 等） —— 本项目不提供
上板验证	未运行	需要实际硬件与测试装置 —— 本项目不提供

综合单元统计示例：PWM 5 个通用门级单元；UART 29 个；FIFO 34 个。

综合通过 ≠ 时序收敛 ≠ 能上板；本项目不做时序签核，也不给出频率结论。

图 3 分层证据

按需追加 Yosys 综合：98 个 RTL 变体（15 参考 + 83 缺陷）全部可综合，并给出通用门级单元统计（PWM 5 个、UART 29 个、FIFO 34 个）；复现见 §9.9。这张表刻意把"没做的"写出来：时序分析、布局布线/比特流、上板验证三行标"未运行"，读者就不会把"综合通过"误读为"能上板"。

一个值得记录的实测结论： $\#5\ q \leq d$ ；这类延时不会让 Yosys 报错——它被静默忽略。所以"综合通过"也不能反过来当作"这段代码的时序语义被正确实现了"的证据。

5.6 外部模块验证（别人写的 RTL，不是我们自己的）

内置基准是我们自己写的，容易被质疑"是不是照着自家测试台长的"。因此补了一组外部模块：alexforencich/verilog-uart（MIT，Verilog-2001，单时钟），接收侧 uart_rx 与发送侧 uart_tx。

流程与内置案例完全一致：人工合约（显式写明复位为高有效 + 同步、prescale 为 16 位）→ 激励 → Icarus → 行为对比。此外每个模块配一份手写规格测试台（只依据 UART 规范，不看 DUT 内部实现）：uart_rx 用"一帧好帧 + 一帧停止位为 0"共 11 项检查；uart_tx 用环回（tx 发、rx

收，收到的字节必须等于发出的) 共 10 项检查。

模块	功能变体	verify-diff 判"不同"	规格测试台抓到	等价改写	编不过的输入
uart_rx (握手 + 状态机)	5	5/5	5/5	判"一致"	判"未取得可比证据"
uart_tx (握手 + 状态机)	5	5/5	3/5	判"一致"	判"未取得可比证据"
priority_encoder (参数化输入, 纯组合)	5	5/5	5/5	判"一致"	判"未取得可比证据"

合计 15 个功能变体, verify-diff 判"不同" 15/15; 三个基线全部通过规格测试台; 三个等价改写全部判"一致" (无误报); 三个编不过的输入全部判"未取得可比证据" (没有被误判成"行为不同")。第三个模块是纯组合逻辑、没有时钟也没有复位, 行为由 WIDTH 与 LSB_HIGH_PRIORITY 两个参数决定——规格测试台里同时实例化两个参数取值, 直接检验"参数化"本身; 合约里不写 clock/reset, 不写等于"没有", 而不是猜一个出来。

两个判据的盲区不同, 正好互补——这是这次外部验证最有价值的一条观察:

环回测试台漏掉"位周期少一拍": 接收侧是 8 倍过采样, 能容忍 ± 1 拍的波特率误差, 环回照样通; 而 verify-diff 把 txd 波形与基线逐拍比对, 立刻看出不同。

环回测试台漏掉"busy 信号不清": 我们的检查只覆盖了"发送中 busy 为 1", 没覆盖"发完归 0"; verify-diff 把 busy 当输出端口比对, 直接抓到。

反方向: 这一批里 verify-diff 没有漏的, 10/10 全部判"不同"。

如实写三条限制: (1) 两个模块来自同一个上游仓库, 只覆盖"握手 + 有限状态控制", 参数化输入一类尚未覆盖; (2) 变体由开发者本人构造, 未经独立复核人审核, 因此只能称为"人工变异", 不能称为"真实社区漏洞"; (3) 每个模块只跑了一个激励计划, 本结果不能外推为"能检出这些模块的所有缺陷"。

外部验证还暴露了两个测试设计层面的坑, 我们都记了下来: 测试字节用了位序回文 0xA5 (正反读一样), 于是"数据位序写反"的变体在行为上与基线完全相同; 以及测试台 @(posedge clk); tvalid = 1'b0; 与 DUT 抢同一个时钟边沿, 导致 DUT 根本没收到数据。两处都是先得到错误结论、再定位修好的——这正是"证据要能复查"的意义: 如果只记录"检出率 100%", 这两个坑会一直藏在测试台里。

证据: `.iverilog-ai/external/uart_rx_check/evidence.json`; 复现命令 `python scripts/fetch_external_modules.py + python scripts/check_external_module.py`; 口径与来源见 `docs/experiment/external_results.md` 与 `docs/experiment/metric_inventory.md`。

六、创新性与开放价值

6.1 创新性

与"让大模型直接写 testbench"的常见方案相比, 本项目的实质改进有三条:

(0) 与已有方案的对比选型

方案	谁在判对错	不需要用户写 TB	判定可复现	成本	我们相对它的差异
人工手写 testbench	人 + 仿真器	✗	是	每个设计 80 行起	我们用同一套判据免掉写 TB，但保留人工边界激励的位置（消融实验显示它仍更强）
通用大模型直接写 TB	模型自己（它同时给期望值）	✓	否	低	判决权从模型手里拿走：AI 写的期望值被参考模型复算覆盖，不一致只记诊断
商业仿真器的 AI 助手	仿真器	部分	是	授权费 + 闭源	我们全链路可审计（判决路径、四层结论口径、逐条证据），且离线可跑、零授权成本
形式化等价验证	求解器	✓	是	需形式化建模	我们不做等价证明，只证明“在测试计划覆盖范围内行为一致”——门槛与成本低一个量级
本项目	开源仿真器 Icarus + 确定性参考模型	✓	离线固定 seed 或重放已保存计划时可复现	离线无模型调用费；在线另计	AI 参与规划、确定性执行裁决，并披露支持范围

把"AI 自评"从判决链里彻底移除。判决权归开源仿真器，AI 只提出假设；AI 给出的期望值会被确定性参考模型复算覆盖。这不是提示词技巧，而是架构上的分工。

证据分层与负向声明。主动写出未做的证据层级（时序/比特流/上板），并把"综合通过 ≠ 时序收敛 ≠ 能上板"作为固定免责声明写入报告与网页。

噪声校准与失败公开成为交付物的一部分。静态规则与波形分析都公开了误报校准过程；在线模型实验公开了 9 个未检出缺陷及逐条原因，而不是只报最佳成绩。

6.2 开放成果

成果	内容	许可
代码仓库	完整工具链（AI 规划层、执行层、证据层、网页）+ 可复用 GitHub Action	Apache-2.0
基准集	15 个案例 / 83 个缺陷变体 + 边界 testbench + 清单	Apache-2.0

規則包	44 條靜態規則（含逐條正反例） + 按案例組織的模型約束規則	Apache-2.0
測試計劃格式	結構化 JSON 合約，可被其他工具或仿真器适配	Apache-2.0
中文文檔	架構、規則清單、波形判據、對齊方法、分層證據、行為對比、10 分鐘複核指南	Apache-2.0

6.3 社區貢獻與复用

如實說明：目前尚無向 Icarus Verilog 上游提交的 Issue 或 Pull Request。本項目是圍繞上游的非官方擴展，與上游維護者無隸屬或背書關係，也不修改、不分發上游代碼。

可复用方式（不需要上游改動）：① 基準集覆蓋計數器、狀態機、FIFO、UART、SPI、握手、去抖、PWM、復位同步等常見模塊的 83 個缺陷變體，可直接用於評測其他驗證工具或教學；② 測試計劃 JSON 格式與"AI 提出、工具裁決"的分工可遷移到其他開源仿真器（Verilator、GHDL 等），換掉執行層即可；③ GitHub Action 讓上游項目在 PR 上直接獲得行為回歸結論（docs/upstream/verify-diff-action.md）；④ 離線調試接口讓任何人在無密鑰、不聯網的環境下復現全部流程與實驗。

6.4 維護計劃

短期繼續擴展案例與缺陷類型（優先協議類與跨時鐘域類）；中期把更多參考模型納入逐拍對齊、為其他開源仿真器提供執行層适配；長期把基準集作為獨立評測資源向上游社區介紹。問題反饋 走倉庫 Issue 入口。

七、合規、安全與風險說明

7.1 開源及第三方許可合規

類別	關鍵義務	我們怎么做
GPL-2.0-or-later 工具（Icarus、GTKWave）	分發衍生作品時須提供對應源碼	以獨立進程調用：不修改、不複製、不重新分發、不鏈接，因此不觸發源碼提供義務；不聲稱上游背書；倉庫不含其任何源碼或二進制
ISC 工具（Yosys / YoWASP）	保留版權與許可聲明	同上（獨立進程調用），聲明見 THIRD_PARTY.md
MIT / BSD / Apache-2.0 / PSF 依賴	保留版權與許可聲明	逐項列在附件《開源及第三方資源使用清單》與 THIRD_PARTY.md
Anton 字體（SIL OFL 1.1）	隨分發保留版權與字體許可	網頁隨包包含 Anton.ttf 及 OFL 文本；與獨立安裝的 EDA 工具分開披露，詳情見開源清單、THIRD_PARTY.md 與 NOTICE
"公開可訪問 ≠ 有再利用授權"	無明確許可不得复制/訓練/再分發	未使用任何"可訪問"但無明確許可的第三方數據集、圖片或音視頻；報告插圖由自有腳本程序化生成

7.2 数据、权利与隐私

未使用任何第三方数据集；不收集用户数据；不涉及个人信息。用户上传的自定义 RTL 默认只在本机处理，不自动外发。常规计划与建议请求发送设计名、验证目标、DUT 合约摘要或脱敏后的失败字段；规则修复建议默认不含源码，只有用户显式勾选“同时发送相关代码片段”后才发送相应片段。请求正文不包含本地文件路径、命令或密钥。用户自带密钥，仓库不打包、不绑定任何模型服务。

7.3 AI 输出、知识产权与内容安全

风险	应对措施
模型幻觉（杜撰端口、位宽、期望值）	四重门控：Schema 校验 → 合约匹配 → 参考模型复算覆盖 → 仅允许模板化断言
提示注入 / 代码注入	模型输出不进入命令、路径或代码；生成器不做字符串拼接；执行不经 shell
错误的“通过”结论	判决权归 Icarus；参考设计出现失败时单独计为误报指标，不计入检出
凭据泄露	默认离线、无密钥；密钥只存在于进程内存或页面会话；.gitignore 排除 .env；仓库检索无密钥
AI 生成内容的权利	不主张为团队自主知识产权；已说明参与环节与团队验证工作

7.4 技术限制与不宜使用的场景

仿真通过 不等于 可综合；综合通过 不等于 时序收敛或能上板；
不做 时序签核、不做 代码覆盖率、不做 CDC 形式化验证；
静态规则是保守的 lint 式检查（不做完整语法树解析，对宏展开与 generate 内动态结构判断能力有限，策略是“宁可漏报也不误报”）；波形分析只覆盖 VCD 中 dump 出来的信号；
参考模型只覆盖内置案例；自定义设计的期望值来源只能是 AI 或人工，工具会显式标注来源、可信度更低；自定义 RTL 不是安全沙箱，请勿用于执行来路不明的设计。

八、总结与展望

8.1 项目总结

本项目围绕 Icarus Verilog 构建了一套"AI 提出、开源工具裁决"的验证工具链：可运行的命令行与网页；15 案例 / 83 缺陷基准集（实跑 83/83 检出、0 误报、0 不可判定）；15/15 参考模型逐拍对齐；44 条静态规则（逐条正反例 + 噪声校准）；波形语义分析、分层证据、行为对比三套深度能力；三组离线消融实测与并集推导检验了规则激励与参考模型的作用，真实 LLM 的结果单独披露；当前回归结果见 docs/experiment/local_validation_2026-10-04.md。离线确定性流程可复现；在线计划生成依赖外部模型，须保留计划和响应以便重放执行。

主要价值：把"AI 辅助验证"从"相信模型"变成"用开源工具裁决模型提出的假设"，并把这个

过程中的失败、误报与边界如实公开——包括那条对我们不利的结论（规划器没有多检出任何一条人工边界激励覆盖不到的缺陷）。

未完成事项：时序与上板证据层（需器件库与硬件，超出本周期条件）；上游社区贡献；更广泛的第三方工具适配；对象③④的用户调研。

团队收获：理解了 IEEE 1364 非阻塞赋值的实际影响（8 个模型错误中有 3 个源于此）；学会把“工具报出的数字”与“数字里有多少是真的”分开评估；体会到“如实说明没做什么”比“多报一个指标”更能提高可信度。

8.2 后续计划

① 扩展案例与缺陷类型（优先协议类与跨时钟域类），并把参考模型对齐方法固化为可复用流程；② 为其他开源仿真器（Verilator、GHDL 等）提供执行层适配，使基准集与测试计划格式可跨工具复用；③ 完善教程与在线演示，降低他人复用门槛，并在权利边界清晰的前提下向上游社区介绍本项目与基准集。

正文完。以下为附录（不计入 15 页正文预算）， 按需提供。

九、附录

9.1 开源及第三方资源使用清单

见随报告提交的附件《开源及第三方资源使用清单》（仓库内：
docs/competition/opensource_resource_list.md）， 已按官方参考表列序编制。

9.2 仓库链接与运行说明

```
# 1. 安装 ( 仅需 Python 3.11+ 与 Icarus Verilog )
python -m pip install -e .

# 2. 命令行快速验证 ( 无需任何密钥、无需联网 )
python -m iverilog_ai run `
--rtl rtl/mod10_counter.v `
--testbench tb/tb_mod10_counter.v `
--top tb_mod10_counter `
--iverilog D:\iverilog\bin\iverilog.exe `
--vvp D:\iverilog\bin\vvp.exe

# 3. 一键复现基准矩阵结论 ( 15 参考 + 83 缺陷 )
python scripts/run_benchmark_matrix.py

# 4. 启动网页演示
streamlit run ui/app.py

# 5. 无密钥、不联网地跑通完整 AI 流程
python -m iverilog_ai.ai.debug_server # 另开一个终端
```

推荐给评委的五分钟复现路径：跑第 3 步看基准矩阵结论 → 跑第 5 步 + 网页看 "AI 生成计划 → 受控生成 testbench → Icarus 裁决 → 报告"的完整链路 → 在网页上 切换到"自定义 RTL"体验导入与行为级对比。

9.3 测试记录与实验记录

记录	位置
基准矩阵逐项结果	仓库 .iverilog-ai/benchmark-matrix/
固定/随机/AI 策略公平实验	docs/experiment/
在线真实模型实验 (含未检出缺陷逐条原因)	docs/experiment/online_model_experiment_2026-09-10.md
参考模型逐拍对齐方法与踩坑记录	docs/reference_model_alignment.md
静态规则清单与噪声校准	docs/static_rules.md
波形分析判据与三轮校准	docs/vcd_analysis.md
分层证据口径与实测	docs/layered_evidence.md
行为级对比判据与实测	docs/behavior_compare.md

9.4 插图说明

本报告插图（图 1–图 4）由团队自有脚本 scripts/make_report_figures.py 使用 Pillow 程序化生成，不包含任何第三方图片素材；脚本可重跑，图与仓库数据保持一致。

9.5 参考文献

IEEE Std 1364-2005, IEEE Standard for Verilog Hardware Description Language.

Icarus Verilog 项目文档与用户指南，<<https://steveicarus.github.io/iverilog/>>。

Yosys Open SYNthesis Suite 文档，<<https://yosyshq.readthedocs.io/>>。

LowRISC Verilog Coding Style Guide（仅参考公开文档中的编码与复位约定，未复制代码）。

cocotb 文档（验证工程实践参考，未复制代码）。

9.6 三个真实事故的完整复盘（正文 §4.2（6）的展开）

#	症状（用户原话 / 现象）	根因（代码位置）	修复	回归测试
1	选「离线」模式、案例选简单 ALU，点"执行 AI 计划并生成 testbench"报 vectors[0].inputs contains unknown port 'rst_n'；同一个案例点"执行真实 Icarus 仿真"却通过，看起来像离线模式坏了	离线分支直接把 MockProvider() 的默认返回值当计划，而那份写死的演示计划固定驱动 rst_n——组合逻辑案例的合约里既没有时钟也没有复位	新增 offline_provider(contract)：按当前合约生成激励（与本地调试模型服务同一个引擎），组合与时序案例都成立	tests/core/test_offline_ui_plan.py：对每个内置案例都要求离线路径能产出可生成的计划
2	配置好 API 后点"从表格生成 JSON"→刷新表格→点"校验 contract"，报 st.session_state.custom_contract_json_text cannot be modified after the widget with key custom_contract_json_text is instantiated	Streamlit 的 widget 一旦实例化就不允许再写同名 session_state 键；而"把规范格式写回文本框"必然要写它	把这两个动作移到 on_click 回调（回调在 fragment 主体之前执行），并去掉所有 st.rerun(scope="fragment")	tests/core/test_ui_smoke.py 加了 AST 门禁：禁止在这两处出现"先建控件再写同名键"的模式
3	生成测试计划报 assertions[0] contains unsupported field(s): signal；再点一次又能生成	提示词里给 signal_implies 列了 signal 字段，而校验器不接受——提示词与校验器各写了一份字段表；重试时换条路径恰好避开	字段表收成单一事实来源 ASSERTION_FIELDS（校验器、执行器、提示词共用）；重试时把拒绝原因追加进提示词	tests/core/test_labels.py 的"单一来源"门禁 + 断言模板门禁

9.7 阶段时间线（正文 §4.3 的展开）

阶段	主要工作	留下的可核验物
一	执行器与安全边界；固定矩阵基准成型	路径策略、进程超时、基准矩阵脚本
二	AI 计划 Schema 与确定性 testbench 生成	四门校验链、结构化记录格式
三	参考模型与逐拍对齐（发现 8 个非阻塞赋值语义错误）	15/15 对齐测试
四	真实模型公平实验（暴露"AI 期望值不可信"）	在线实验记录与口径更正
五	引入权威预言机覆盖，参考误报归零	参考模型复算 + 三态证据等级
六	静态规则与波形判据的三轮校准	44 条规则、逐条正反例
七	面向两类主打对象适配：verify-diff、GitHub Action、批量批改	消融实验、Action、复用文档
八	诚实性加固：四层结论口径、提交体检、10 分钟复核指南	措辞门禁、占位符门禁、复核指南

9.8 消融实验中规划器漏检的 7 条（正文 §5.3（2）的展开）

案例	缺陷	触发条件	通用激励为什么碰不到
mod10_counter	mod10_bug_async	复位在两沿之间断言	需要复位释放与时钟沿的精确相位关系；整拍边界给复位看不出异步与同步之差
sync_reset	srst_bug_single_stage	ext_rst_n 释放	差别只在释放路径的同步级数上，需要释放时刻恰好落在时钟沿附近
sync_reset	srst_bug_sync_assert_only	ext_rst_n 在两沿之间拉低	断言路径的同步与否只在释放窗口里可观测
debounce	deb_bug_counter_not_cleared	输入短暂回到采样值后再抖动	需要足够长的连续抖动序列才能让计数器累计出错
pulse_stretcher	pstr_bug_no_retrigger	展宽期内再次给脉冲	需要在展宽期内精确重触发，通用激励没构造重叠时序
sync_fifo	fifo_bug_full_off_by_one	写满 DEPTH 个数据	写入长度没卡在满/空边界上
traffic_light_emergency	traffic_bug_emergency_output	emergency=1	抢占时刻没落在那个状态拍上

9.9 可复现命令清单

想验证什么	命令
参考设计全过、缺陷全检出、零误报	python scripts/run_benchmark_matrix.py
离线规则计划与参考模型消融（三组实测）	python scripts/run_ablation_matrix.py

98 个变体全部可综合	python scripts/run_synthesis_matrix.py
离线 AI 路径 15/15	python scripts/run_pipeline_matrix.py
实验费用区间（按核验过的官方价目表）	python scripts/estimate_model_cost.py <实验目录>
全部自动化测试与类型门禁	python -m pytest -q / python -m mypy
两份 RTL 行为对比（不需要合约与计划）	python -m iverilog_ai verify-diff --baseline a.v --candidate b.v
批量批改一个目录	python -m iverilog_ai grade --dir submissions/ --golden rtl/ref.v
提交材料体检（页数 / 体积 / 元数据实名 / 占位符）	python scripts/check_submission.py report.pdf / --repo .

9.10 完整测试条件（正文 §5.2 的展开）

项	条件
仿真器	Icarus Verilog 12.0 (iverilog + vvp)，本机进程调用，-g2012
综合器	Yosys（可选层，不参与 PASS/FAIL 裁决）
运行平台	Windows 11 x64 + Python 3.11；CI 另跑 Linux（Ubuntu）与 Python 3.11/3.12
判决口径	只有 Icarus 的编译/仿真结果与结构化断言参与裁决；AI 的数字只作诊断
期望值口径	内置案例由参考模型复算覆盖（reference_model）；自定义 RTL 回退并如实标注
随机性	离线固定 seed=0 与固定激励可确定性重放；随机策略记录 seed；在线模型重新生成计划可能不同，已保存计划可重放执行
单例超时	30 秒（综合层 180 秒）；超时记为"不可判定"，不折算成检出
复现入口	python -m pytest -q 与 5 个矩阵脚本（§9.9）；离线路径不需要密钥或联网，在线模型实验需要模型服务

9.11 静态规则与波形判据的三轮校准（正文 §5.4 的展开）

静态规则：初版 9 条规则、对仓库 84 个 RTL 文件报出 317 条命中，逐条核实后修掉：
 division-operator 把注释结尾 / 当成除法（84/84 全命中）、inferred-latch 把合法写法
 误判为锁存器（15 条 error 级误报）、missing-timescale 因注释剥离而全命中、时钟/复位名
 只匹配前缀（rst）导致实际命名 rst_n 从不命中。校准后：44 条规则、163 条命中、0 条 error
 级误报。

波形毛刺判据：初版按翻转次数误报时钟与计数器；第二版用“短于全局中位数 1/3”误报 busy
 的正常窄脉冲。第三版采用全局基线、连续 ≥ 3 个异常间隔并排除复位沿，在已测时钟、
 计数器、脉冲与双峰样例上无误报，连续快翻仍可命中。